

RETO 2 — ACTIVIDAD 1

Análisis de puntos de dolor de la arquitectura actual

Medición sobre el código entregado en el Reto 1 (03-src/SolucionFarmacia/). El costo se expresa contando las clases y los archivos que hay que abrir para atender cada escenario de cambio; las referencias de línea corresponden al estado actual del código.

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
P-01	Clases/MedicamentoCapsula.cs, Clases/MedicamentoLiquido.cs, Clases/ProductoRequest.cs:15-22, Factories/FabricaProducto.cs:14-36, Repositorios/RepositorioProductos.cs:60-66, AppFarmaciaConsola/Program.cs:375-387, 434, 458, 501	La forma en que viene presentado un medicamento — cápsula con relleno, líquido con envase— está modelada creando una subclase por presentación. Esa elección se repite escrita de cuatro maneras distintas: lista blanca de tipos, switch sobre Type, switch sobre la cadena del archivo, e if con typeof en el menú. Una presentación nueva obliga a crear una subclase y a tocar los cuatro sitios. Además, los atributos que justifican esas subclases (Relleno, Envase, Mililitros) no los lee ningún consumidor: sus únicas apariciones fuera de las propias clases son las tres líneas de la fábrica que los pasa al constructor.	4 clases a modificar (ProductoRequest, FabricaProducto, RepositorioProductos, Program) y 4 archivos a abrir, más 1 clase nueva por cada presentación que se agregue.	Alta — Propio
P-02	Clases/Cliente.cs:28-33, Servicios/ServicioPuntos.cs:21-30, AppFarmaciaConsola/Program.cs:665-676	La única manera de que un cliente gane puntos es una suma fija escrita dentro de la propia entidad. No existe ningún lugar donde pueda vivir una regla de acumulación distinta —por promoción, por tipo de cliente o	3 clases a modificar (Cliente, ServicioPuntos, Program) y 3 archivos a abrir, más una modificación adicional en los mismos tres	Alta — Propio

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
		por monto de compra—. De hecho, la regla de cuántos puntos genera una compra no existe en ninguna parte: el operador teclea la cantidad a mano en el menú. Ofrecer una segunda forma de acumular obliga a modificar la entidad, el servicio y el menú a la vez.	por cada forma nueva de acumulación.	
P-03	Interfaces/IDescuento.cs:11, Servicios/ServicioDescuento.cs:15, Clases/Cliente.cs, Repositorios/RepositorioClientes.cs	El contrato de descuento recibe un número y nada más: <code>CalcularDescuento(decimal precio)</code> . Un descuento que dependa de quién compra no se puede expresar con esa firma, ni siquiera implementándola bien. El único descuento que existe devuelve un 10 % escrito directamente en el código. Y el cliente no tiene ningún dato de afiliación, así que tampoco hay a qué asociarle un convenio. Seguir el molde actual significa una clase por convenio, cada una con su porcentaje fijo y ninguna capaz de mirar al cliente.	4 clases a modificar (<code>IDescuento</code> , <code>ServicioDescuento</code> , <code>Cliente</code> , <code>RepositorioClientes</code>) y 4 archivos a abrir, más 1 clase nueva por cada combinación de entidad y beneficio: con las cinco entidades y los dos beneficios del Anexo B, hasta 10.	Alta — Propio
P-04	Servicios/ServicioFacturacion.cs:13-19, Servicios/ServicioVenta.cs:31-53, AppFarmaciaConsola/Program.cs:350	El único cálculo de dinero del sistema es <code>precio × cantidad</code> en dos líneas, sin ningún punto donde insertar algo antes o después. Y la venta ni siquiera lo usa: <code>RegistrarVenta</code> descuenta existencias y registra el movimiento sin calcular importe, mientras que <code>CalcularTotal</code>	3 clases a modificar (<code>ServicioFacturacion</code> , <code>ServicioVenta</code> , <code>Program</code>) y 3 archivos a abrir.	Alta — Asistido

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
		se invoca una sola vez, desde "Ver movimientos", o sea después de vender y solo para mostrar. No hay momento en el flujo donde se decida cuánto se cobra, de modo que no hay dónde aplicar dos ajustes sucesivos sobre el mismo importe.		
P-05	Factories/FabricaProducto.cs:14-36, Repositorios/RepositorioProductos.cs:58-76, Clases/ProductoRequest.cs:15-22, AppFarmaciaConsola/Program.cs:434-516	La primera columna de productos.txt es hoy la única fuente de la decisión de qué se construye, y se interpreta en tres sitios distintos. Al resolver P-01 esa decisión desaparece junto con el único lugar que la alojaba. Cuando entren categorías que sí son clases realmente distintas — un cosmético o un paquete de comestibles no tienen laboratorio ni presentación—, la decisión reaparece sin sitio asignado y vuelve a repartirse entre el repositorio, la solicitud de creación y el menú. El sistema no tiene un lugar designado donde se decida qué clase de producto nace.	4 clases a modificar (ProductoRequest, FabricaProducto, RepositorioProductos, Program) y 4 archivos a abrir, más 1 clase nueva por cada categoría de producto.	Media — Asistido
P-06	Interfaces/IFacturable.cs:14, Clases/Producto.cs:27-33, 72-75, Clases/Servicio.cs:50-53, Servicios/ServicioFacturacion.cs:13-19	El precio de lo que se vende es un campo fijo. ObtenerPrecio() está declarado en la interfaz como el lugar donde cada cosa vendible decidiría su precio, pero las dos implementaciones devuelven el campo tal cual y el método no es sobrescribible, así	3 clases a modificar (Producto, Servicio, ServicioFacturacion) y 3 archivos a abrir, más 1 clase nueva por cada tipo de promoción.	Media — Propio

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
		que ningún subtipo puede cambiar esa regla. Una promoción sobre un producto concreto solo se puede hacer modificando la clase raíz del catálogo o alterando el Precio del objeto, que es un cambio permanente y afecta también al listado de productos y a las alertas. El archivo de datos tampoco tiene dónde declarar qué productos están en promoción.		
P-07	AppFarmaciaConsola/Program.cs:251-703	El menú es un switch de once casos donde cada opción lleva su propio bloque de lectura de teclado, y ocupa 452 de las 707 líneas del archivo. Una opción nueva agrega otro caso al mismo sitio. No se interviene: son ramas de captura de entrada, no de negocio; el encargo limita el trabajo a la colaboración entre objetos del back; y sustituirlas exigiría once clases nuevas para un menú que no crece con ninguna de las tres solicitudes del Anexo B. El remedio cuesta más que el problema.	1 archivo, y 1 caso nuevo por cada opción que se agregue — solo si se decidiera intervenir	No se interviene — Propio
P-08	BibFarmacia/Interfaces/IServicioNotificacion.cs, BibFarmacia/Servicios/ServicioNotificacion.cs, cableado de eventos en AppFarmaciaConsola/Program.cs:91-136	Existe un servicio de notificación que escribe directamente en consola, pero ningún flujo lo utiliza: cero referencias externas a la interfaz o a la clase. Las notificaciones reales se resuelven por otro camino, con las cuatro	Para conectar un canal de notificación al flujo de puntos o ventas: 1 clase de canal nueva y 2 clases existentes modificadas (ServicioNotificacion y Program), 3 archivos	Media — Propio

ID	Dónde (archivo / clase)	Qué lo hace rígido o caro	Cuánto cuesta hoy	Prioridad y origen
		clases Evento* (delegado, evento, Disparar()), cuyos manejadores también están conectados directamente a la consola. Hay dos mecanismos conceptuales para lo mismo, sin un límite claro entre evento de dominio y canal de salida.	como mínimo. Si además se amplía la interfaz para soportar destinatario, canal y tipo de evento, se suma IServicioNotificacion, totalizando 4 archivos.	

Notas de la medición

1. Los archivos de datos (productos.txt, clientes.txt, servicios.txt, usuarios.txt) se consideran congelados: ninguno de los costos anteriores incluye cambiar su formato, y las soluciones que se evalúen deben poder aplicarse sin tocarlos.
2. P-01 y P-05 son consecutivos, no simultáneos: P-05 describe el hueco que queda una vez resuelto P-01, y por eso su prioridad es menor hoy. Pasa a Alta si el equipo implementa la solicitud de cosméticos y comestibles.
3. P-03, P-04 y P-06 tocan el precio, pero son problemas distintos y se separan por eje. P-03 es que no se puede expresar **de quién** depende un descuento: falta el dato de afiliación del cliente y falta el vínculo entre la venta y quien compra. P-04 es que no existe un **momento** en el flujo donde se calcule el importe que se cobra. P-06 es que no hay dónde colgar una condición sobre **qué** se vende, porque el precio es un campo fijo y su lectura no es sobrescribible. En P-06, a diferencia de los otros dos, el dato y el vínculo ya existen: la venta sí conoce el producto que se está vendiendo. Resolver uno no resuelve los otros.

RETO 2 — ACTIVIDAD 2

Entregable 2.1 — Tabla de decisión de patrones

Cada patrón evaluado queda anclado a un punto de dolor concreto del análisis de la Actividad 1. Las referencias de línea corresponden al código entregado en el Reto 1 (03-src/SolucionFarmacia/).

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
Strategy	Comportamiento	P-02 — La única forma de que un cliente gane	Gana: la regla de acumulación deja de estar fija dentro de Cliente y	Adoptado	Existe una operación con varias formas posibles y ninguna razón para

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
		puntos es una suma fija escrita dentro de la propia entidad	pasa a ser un objeto sustituible; agregar una promoción es una clase nueva más una línea en el punto de ensamblaje, sin tocar la entidad, el servicio ni el menú. Cuesta: una interfaz y una clase por cada regla, y la regla queda fija durante toda la sesión, de modo que cambiar la promoción a media jornada obliga a reiniciar la aplicación.		privilegiar una. Hoy la única forma es la suma fija de <code>Cliente.cs:28-33</code> , y ofrecer una segunda obliga a modificar la entidad, el servicio y el menú a la vez. La regla por defecto reproduce exactamente la suma actual, así que el comportamiento observable no cambia. El administrador configura la promoción del día inyectando la regla en <code>Program.cs:62-63</code> , con el mismo mecanismo que el sistema ya usa para rellenos y envases en <code>Program.cs:74-89</code> .
Decorator	Estructural	P-02 — Mismo punto: no hay dónde vivan reglas de acumulación distintas	Gana: permitiría apilar reglas —una base más una promoción— sin escribir una clase por cada combinación. Cuesta: una capa de envoltura permanente y un recorrido más difícil de depurar, porque hay que seguir la cadena para saber de dónde salió un valor.	Descartado	Decorator resuelve combinar reglas, no elegir entre ellas, y hoy no existe el escenario que resuelve: el operador teclea un número y una sola regla lo transforma. Ninguna de las tres solicitudes del Anexo B plantea promociones simultáneas. Con Strategy, una combinación futura se resuelve escribiendo una clase que haga las dos cosas; sostener una capa de envoltura permanente para ahorrar una clase que hoy no existe cuesta más de lo que ahorra. Se suma un riesgo propio de este código: <code>Cliente.AcumularPuntos</code> (<code>Cliente.cs:28-33</code>) exige puntos positivos, y una envoltura que multiplique por una fracción puede redondear a cero y romper esa precondition sin aviso.

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
State	Comportamiento	P-02 — Mismo punto, visto como niveles de cliente	Gana: modelaría niveles de cliente en los que acumular puntos cambia el nivel y el nivel cambia la forma de acumular. Cuesta: una clase por nivel más la lógica de transición, y obligaría a agregar a <i>Cliente</i> un atributo de etapa que hoy no existe.	Descartado	State no consiste en elegir entre opciones: consiste en que el objeto cambie de comportamiento como consecuencia de su propio ciclo de vida, gobernando además las transiciones. Aquí no hay ciclo de vida que modelar: un cliente no transita entre modos de acumulación y <i>Cliente</i> no tiene ningún atributo que represente una etapa. Adoptarlo exigiría inventar niveles de cliente que ningún requisito del enunciado menciona.
Strategy	Comportamiento	P-01 — La presentación de un medicamento (cápsula con relleno, líquido con envase) está modelada creando una subclase por presentación, con la decisión de tipo repetida en cuatro sitios	Gana: un único punto donde ambas presentaciones se resolverían por polimorfismo en vez de por subclase. Cuesta: forzar que <i>IRelleno</i> e <i>IEnvase</i> compartan un contrato que hoy no tienen, o inventar una interfaz artificial que envuelva a las dos.	Descartado	Strategy exige que las implementaciones intercambiables compartan un contrato común. <i>IRelleno</i> (<i>Nombre</i> , <i>InstruccionesConservacion</i> ()) e <i>IEnvase</i> (<i>Material</i> , <i>EsRetornable</i> ()) son interfaces distintas sin métodos en común: no existe una sola "estrategia de presentación" que a veces sea una y a veces la otra. Además, Strategy ya resuelve, desde el Reto 1, la variación dentro de cada presentación —qué relleno, qué envase—; lo que P-01 describe es otro problema: cuál subclase se instancia, que es una decisión creacional, no de comportamiento.
Decorator	Estructural	P-01 — Mismo punto: la presentación se modela como subclase en vez de como composición	Gana: construir un <i>Medicamento</i> base y envolverlo con la presentación, sin subclase. Cuesta: dos decoradores, uno por	Descartado	<i>Medicamento</i> es una clase concreta (<i>Clases/Medicamento.cs:9</i>), así que envolverla es técnicamente posible. Pero <i>Decorator</i> existe para

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
			presentación, que en la práctica serían las dos clases de hoy con otro nombre, más el mecanismo de envoltura.		combinar comportamientos que pueden coexistir, y un medicamento nunca es cápsula y líquido a la vez: es una elección excluyente, no una suma. Es el mismo argumento con el que se descartó Decorator en P-02 — sin necesidad de apilar, la envoltura no aporta nada que la composición directa no dé.
Simple Factory (evaluado primero como Factory Method)	Creacional	P-01 — La decisión de qué subtipo construir estaba repetida en <code>ProductoRequest</code> , <code>FabricaProducto</code> , <code>RepositorioProductos</code> y <code>Program.cs</code>	Gana: ningún consumidor hace <code>new MedicamentoCapsula/new MedicamentoLiquido</code> directamente — la construcción concreta queda detrás de <code>IFabricaMedicamentos.Crear</code> , con un método por presentación en vez de un constructor expuesto. Los cuatro sitios de decisión del Reto 1 quedaron en uno solo (<code>RepositorioProductos</code>). Cuesta: ese único sitio sigue siendo un switch — agregar una presentación de medicamento todavía obliga a abrir <code>RepositorioProductos.cs:58-79</code> , así que el costo de OCP que P-01 medía se acotó, no desapareció.	Adoptado	Se evaluó e implementó primero Factory Method : un creador por tipo (<code>ICreadorMedicamento</code> , <code>CreadorCapsula</code> , <code>CreadorLiquido</code>) seleccionado por un registro en el <code>composition root</code> , sin ningún switch. Se verificó en ejecución — los diez productos de <code>productos.txt</code> cargaron con nombre, stock y precio idénticos a los de antes del patrón. El equipo decidió no mantener esa versión y quedarse con la fábrica de sobrecargas que ya existía, con este argumento: el eje real de extensión de negocio —categorías de producto nuevas, como los cosméticos y comestibles de SC-1— nunca va a pasar por <code>RepositorioProductos</code> ni por <code>FabricaMedicamentos</code> , porque cada categoría futura tendrá su propio par fábrica/servicio, completo e independiente. El switch que queda solo decide entre las dos presentaciones de un medicamento —cápsula y líquido—, un conjunto cerrado por la

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
					naturaleza del dominio farmacéutico, no un punto que las tres solicitudes del Anexo B vayan a hacer crecer. Es una decisión de simplicidad deliberada, no una renuncia al problema: lo que resultó sí es un patrón real y con nombre — Simple Factory, ampliamente documentado aunque no forme parte del catálogo de cinco creacionales del Anexo A—, y se lo llama así en vez de forzarlo bajo el nombre de otro patrón que no le corresponde. El costo aceptado, y el hallazgo nuevo que salió de volver a auditar el código tras la decisión, están en la Actividad 4: la fábrica de sobrecargas trae de vuelta un acoplamiento concreto en RepositorioProductos que el equipo no había declarado.
Command	Comportamiento	P-03 — El contrato de descuento no puede expresar de quién depende un descuento	Gana: poder encolar, registrar o deshacer la aplicación de un descuento como una operación independiente. Cuesta: una clase de comando por cada tipo de descuento, más la infraestructura para invocarlos, sin que nada en el dominio necesite guardarlos, encolarlos o revertirlos.	Descartado	Command encapsula una acción como objeto para poder posponerla, registrarla o deshacerla; su valor está en el historial y el control de la ejecución, no en variar el resultado de un cálculo. Aquí no hay ningún requisito de deshacer un descuento ni de encolarlo: lo que se necesita es elegir, en el momento de facturar, cuál regla de cálculo aplica según el cliente. Eso es lo que resuelve Strategy, no Command.
Strategy	Comportamiento	P-03 — Mismo punto: el descuento actual es un 10	Gana: cada convenio pasa a ser una implementación intercambiable de	Adoptado	Existe una operación —calcular el descuento— con tantas variantes

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
		% quemado, sin manera de expresar de quién depende	un mismo contrato, sin condicionales por tipo de cliente ni porcentajes quemados. Cuesta: la firma actual, <code>CalcularDescuento(decimal precio)</code> , no alcanza; hay que ampliarla o resolverlo de otra forma para que la estrategia pueda depender de quién compra, y el cliente necesita cargar la referencia a su propia estrategia.		posibles como convenios, y ninguna razón para privilegiar una: es el caso de uso central de Strategy. No se resuelve con sobrecarga de métodos, que se decide en tiempo de compilación por el tipo estático del argumento; se resuelve con polimorfismo en tiempo de ejecución, sustituyendo qué objeto <code>IDescuento</code> tiene inyectado cada cliente. El costo queda declarado: adoptarlo obliga a decidir cómo el descuento llega a conocer al cliente, decisión que se detalla en la ficha del patrón.
Bridge	Estructural	P-08 — Hay dos mecanismos de notificación sin límite claro entre evento de dominio y canal de salida	Gana: separar en dos jerarquías independientes el tipo de evento (stock, vencimiento, puntos, movimiento) y el canal de salida (consola, y en el futuro archivo o correo), combinando cualquier evento con cualquier canal sin multiplicar clases. Cuesta: mantener una segunda jerarquía —la de canales— cuando hoy solo existe uno, la consola.	Descartado	Bridge se justifica cuando dos ejes de variación crecen por separado. Hoy solo uno de los dos varía: los tipos de evento, que ya son cuatro y podrían crecer. El canal de salida es uno solo, y ninguna de las tres solicitudes del Anexo B pide agregar otro. Introducir la jerarquía de canales para desacoplarla de un eje que todavía no existe es generalidad especulativa: se pagaría una abstracción hoy para una variación que no está pedida.
Composite	Estructural	P-08 — Mismo punto: los dos mecanismos de notificación conviven sin límite claro	Gana: tratar una notificación individual y un grupo de notificaciones de forma uniforme, como nodos de un mismo árbol. Cuesta: modelar una jerarquía todo-parte que hoy no existe en ningún lado del dominio.	Descartado	Composite resuelve estructuras parte-todo, donde un objeto compuesto contiene otros del mismo tipo y el cliente los trata igual. Aquí no hay ninguna notificación compuesta de otras notificaciones, ni ningún escenario que pida agrupar

Patrón evaluado	Familia	Punto de dolor que atacaría	Qué gana y qué cuesta	Decisión	Por qué
					avisos y tratarlos como uno solo: cada evento dispara un aviso independiente. No hay jerarquía que modelar.
Observer	Comportamiento	P-08 — El servicio de notificación no lo usa nadie; las notificaciones reales están cableadas directo a la consola	Gana: un límite claro entre qué pasó en el dominio (una venta, una acumulación de puntos) y quién reacciona a eso, sin que quien dispara el evento conozca a quien lo escucha. Cuesta: que <code>IServicioNotificacion</code> y <code>ServicioNotificacion</code> , hoy sin usar, hay que conectarlos de verdad o retirarlos; dejarlos como están sería declarar un patrón que el código no demuestra.	Adoptado	El propio código ya insinúa esta solución: las cuatro clases <code>Evento*</code> (<code>delegado</code> , <code>evento</code> , <code>Disparar()</code>) son <code>Observer</code> aplicado a medias, y hoy sus únicos suscriptores son lambdas de consola cableadas en <code>Program.cs</code> . Formalizar ese mecanismo, en vez de introducir uno nuevo, es la opción de menor riesgo: no se reemplaza infraestructura que funciona, se le da el límite claro que le falta entre evento de dominio y canal de salida.

Nota sobre la aplicación de Strategy en P-02

La regla se inyecta en el punto de ensamblaje, **no se selecciona desde el menú**. El administrador configura la promoción del día antes de arrancar la aplicación, de la misma forma en que hoy se configuran los adaptadores de persistencia. No se agrega ninguna opción visible al menú, de modo que la salida del sistema con la regla por defecto es idéntica a la del sistema anterior.

Nota sobre Simple Factory en P-01 (Factory Method implementado, luego retirado)

Al evaluar la primera versión se descartó expresamente el nombre "Abstract Factory": el sistema tiene una sola familia de productos (medicamentos), y el Anexo A advierte que en ese caso la abstracción adicional de una fábrica de fábricas no se justifica. Lo que resolvía el problema por completo era un creador con una implementación por subtipo — la definición de `Factory Method` — y así se implementó: `ICreadorMedicamento` con `CreadorCapsula` y `CreadorLiquido`, seleccionados por un registro en `Program.cs`, sin `switch` en ningún lado.

Después de implementarlo y verificar que preservaba el comportamiento, el equipo decidió no mantener esa versión y quedarse con `FabricaMedicamentos` — una clase con un método `Crear` sobrecargado por presentación, con el `switch` que decide cuál sobrecarga invocar todavía dentro de `RepositorioProductos`.

Esto **sí es un patrón, pero no es Factory Method**: es Simple Factory, una técnica de creación ampliamente documentada —aparece explícitamente en textos de referencia como paso previo a Factory Method— que no forma parte del catálogo de cinco patrones creacionales del Anexo A. Se la nombra como lo que es, en vez de forzarla bajo el nombre de un patrón distinto, precisamente para no incurrir en la penalización del enunciado por declarar un patrón que el código no sostiene.

El argumento del equipo para preferir Simple Factory sobre Factory Method: el eje de extensión que de verdad les importa —nuevas categorías de producto por SC-1— no pasa por este switch, porque cada categoría futura tendrá su propio par fábrica/servicio. El switch que queda solo decide entre cápsula y líquido, un conjunto cerrado en la práctica. Lo que Simple Factory no resuelve —y Factory Method sí resolvía— es exactamente ese switch: sigue siendo un punto que hay que reabrir si aparece una tercera presentación de medicamento. El costo completo, incluido un hallazgo adicional sobre cómo quedó construida esta versión, está desarrollado en la Actividad 4.

Nota sobre Strategy en P-03

Adoptar Strategy no cierra por si solo el punto de dolor: el contrato actual (`CalcularDescuento(decimal precio)`) no recibe al cliente, así que el descuento correcto no puede resolverse solo con precio. La forma en que la estrategia llega a saber de quien depende el descuento -si el cliente carga la referencia a su propia estrategia, o si se amplía la firma- queda para la ficha del patron (Entregable 3.3), no para esta tabla.

Nota sobre Observer en P-08

Se adopta con una condicion: si `IServicioNotificacion` y `ServicioNotificacion` siguen sin un solo consumidor despues de aplicar el patron, el punto de dolor no quedo resuelto, solo documentado. La demostracion de la Actividad 4 debe mostrar al menos un canal real suscrito a un evento de dominio. RETO 2 — ACTIVIDAD 4

Entregable 4.1 — Matriz de verificación

Una fila por patrón adoptado. Valores: **Refuerza**, **Neutro**, **Tensionado pero compensado**, **Roto**. Toda celda distinta de Neutro lleva su evidencia debajo, referida a archivo y método reales del código en `RetoFarmacia-Patrones/03-src/SolucionFarmacia`.

Nota sobre P-01. Se evaluó e implementó primero Factory Method para este punto — verificado en ejecución, comportamiento preservado —, y el equipo decidió no mantener esa versión. Lo que quedó en `RepositorioProductos/FabricaMedicamentos` **sí es un patrón real**: Simple Factory, una técnica de creación documentada aunque no forme parte del catálogo de cinco creacionales del Anexo A. Se evalúa aquí con su nombre correcto, contra el código que realmente existe hoy — no contra el que se retiró.

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
Simple Factory — P-01 (medicamento)	Neutro	Tensionado pero compensado	Neutro	Neutro	Roto
Strategy — P-02 (puntos)	Refuerza	Refuerza	Neutro	Neutro	Refuerza
Strategy — P-03 (descuento)	Refuerza	Refuerza	Neutro	Neutro	Tensionado pero compensado
Observer — P-08 (notificación)	Refuerza	Refuerza	Neutro	Neutro	Refuerza

Simple Factory — P-01 (medicamento)

SRP — Neutro. FabricaMedicamentos sí tiene una única razón de cambio real —cómo se construye cada presentación de medicamento—, y eso ya era cierto desde antes de esta decisión: ningún consumidor hace `new MedicamentoCapsula` o `new MedicamentoLiquido` directamente. Pero el punto de dolor original de P-01 era `RepositorioProductos`, y ese no cambió de responsabilidad: sigue parseando la línea del archivo **y** decidiendo qué presentación construir **y** proveyendo los valores por defecto de relleno y envase (`RepositorioProductos.cs:52-79`), las tres cosas en el mismo método. La fábrica está bien enfocada; el consumidor que tenía el problema, no.

OCP — Tensionado pero compensado. El switch de datos[0] sigue en `RepositorioProductos.cs:58-79`. Agregar una tercera presentación de medicamento —un tercer formato que no sea cápsula ni líquido— obligaría a reabrir este archivo y agregar un caso más, exactamente el error típico que el enunciado nombra primero: *"una fábrica que crece con un condicional por cada tipo nuevo → movieron el punto de modificación en vez de eliminarlo"*. Es real: el punto de modificación se movió (de cuatro sitios en el Reto 1 a uno solo aquí), no desapareció.

Se compensa con un argumento de alcance, no de diseño: el eje de extensión que las tres solicitudes del Anexo B realmente exigen es categorías de producto nuevas —cosméticos, comestibles—, y esas nunca van a pasar por este switch, porque cada categoría tendrá su propio par fábrica/servicio, completamente aparte. El conjunto que este switch cubre —presentaciones de un mismo medicamento— es pequeño y estable en la práctica farmacéutica. El costo que se acepta, declarado y no oculto: si alguna vez aparece una tercera presentación de medicamento, hay que volver a abrir este archivo.

LSP — Neutro. No hay contrato de sustitución que evaluar. IFabricaMedicamentos declara dos métodos Crear con firmas y tipos de retorno distintos (MedicamentoCapsula en uno, MedicamentoLiquido en el otro) — no hay un único contrato compartido que ambas implementaciones deban cumplir por igual, ni un consumidor que trate a los dos resultados de forma intercambiable. RepositorioProductos llama explícitamente al método correcto para cada caso; no hay polimorfismo que sustituir.

ISP — Neutro. IFabricaMedicamentos tiene dos métodos relacionados —construir cada presentación de medicamento— y la única implementación (FabricaMedicamentos) los cumple ambos de verdad, sin dejar ninguno vacío ni lanzar NotImplementedException. No hay síntoma de interfaz gorda que declarar.

DIP — Roto, declarado sin una compensación real, y con arreglo de una línea. Dos hallazgos concretos:

Primero, RepositorioProductos tiene de vuelta el constructor sin parámetros que hace new FabricaMedicamentos() por dentro (RepositorioProductos.cs:17-20), y es el que usa el composition root (Program.cs:40). Program.cs **también** crea su propia new FabricaMedicamentos() para pasársela a ServicioProducto (Program.cs:41). Son dos instancias distintas de la misma fábrica, y el composition root deja de ser, de hecho, el único lugar donde se construye un concreto — el propio comentario del archivo (Program.cs:36) dice que sí lo es, y hoy no es cierto.

Segundo, RepositorioProductos.Cargar sigue escribiendo new RellenoGel() y new EnvaseVidrio() directamente dentro de su switch (RepositorioProductos.cs:67, 75) — el repositorio conoce las clases concretas de las estrategias, no solo sus interfaces.

No hay un beneficio de diseño que compense esto: es un descuido de conveniencia (el constructor por defecto existe para no tener que pasar la fábrica explícitamente), no una decisión deliberada, y tiene arreglo trivial — que Program.cs pase la misma instancia de FabricaMedicamentos que ya crea, y que los valores de relleno/envase por defecto se resuelvan igual que los del registro de rellenos/envases del propio Program.cs:93-105, en vez de quedar quemados dentro del repositorio. Se deja documentado como hallazgo, no como logro, y como algo que el equipo puede corregir en minutos si decide hacerlo.

Strategy — P-02 (regla de puntos)

SRP — Refuerza. Antes, ServicioPuntos.AcumularPuntos no tenía ninguna regla que sustituir: la cantidad de puntos era exactamente la que llegaba tecleada. Ahora hay tres responsabilidades separadas y cada una vive en un solo lugar: IReglaPuntos.Calcular decide cuántos puntos resultan de una base (ReglaPuntosEstandar, ReglaPuntosDoble); ServicioPuntos.AcumularPuntos solo coordina —pide el cálculo, delega la acumulación, dispara el evento—; Cliente.AcumularPuntos solo valida y suma. Ningún método hace más de una de esas tres cosas.

OCP — Refuerza. ReglaPuntosDoble se agregó sin modificar Cliente.cs, ServicioPuntos.cs ni el menú. Verificado en ejecución: con ReglaPuntosDoble cableada, 50 tecleados dan 100; con la regla estándar, 50 dan 50 — el cambio de comportamiento es exclusivamente el que autoriza la nueva clase.

LSP — Neutro. `Calcular(int) → int` es una función total en ambas implementaciones: no lanzan excepciones, no imponen precondiciones sobre el argumento, y ninguna fortalece ni debilita nada frente a la otra. No hay tensión que declarar, pero tampoco es un punto donde el patrón esté "reforzando" LSP de forma notable — simplemente no hay contrato que romper.

ISP — Neutro. `IReglaPuntos` nace con un solo método. No parte una interfaz gorda preexistente (no existe la responsabilidad `IReglaPuntos.CalcularOtraCosa` que alguien no necesite); es una interfaz nueva ya angosta desde el diseño.

DIP — Refuerza. `ServicioPuntos` depende de `IReglaPuntos` inyectada por constructor (`ServicioPuntos.cs:19-25`), no de una implementación concreta. La regla vigente se decide en `Program.cs` (composition root), con el mismo mecanismo de inyección que ya usa el resto del sistema — `ServicioProducto` con su repositorio, `ServicioAutenticacion` con su autenticador.

Strategy — P-03 (descuento)

SRP — Refuerza. `DescuentoPorcentual` y `SinDescuento` tienen una sola razón de cambio: cómo se calcula un descuento. `ServicioFacturacion` solo resta lo que la estrategia le devuelve. Antes, la única implementación existente (`ServicioDescuento`, hoy eliminada) mezclaba "ser una estrategia de descuento" con "no tener ningún consumidor" — no era una violación de SRP en sentido estricto, pero sí una clase sin responsabilidad ejercida. Ahora cada clase de descuento tiene un rol real.

OCP — Refuerza. Un convenio nuevo es una clase nueva que implementa `IDescuento`; ni `Cliente` ni `ServicioFacturacion` se reabren. Verificado: con `SinDescuento` (valor por defecto), `CalcularTotal` da 10000 sobre `Dolex × 2` — igual que antes de que existiera el patrón. Con `DescuentoPorcentual(0.10m)` cableado sobre el mismo cliente, da 9000.

LSP — Neutro, con una salvedad que conviene declarar aunque no llegue a romper el principio. `IDescuento.CalcularDescuento(decimal precio) → decimal` no declara ningún límite sobre el resultado. `DescuentoPorcentual` no valida su porcentaje en el constructor: un porcentaje mayor a 1 o negativo produciría un descuento mayor al precio o un "descuento" que en realidad incrementa el total. Ninguna de las dos implementaciones actuales viola el contrato de la otra — ambas son funciones totales, sin excepciones sorpresa—, así que no hay sustitución rota hoy. Pero es una invariante que el contrato debería declarar y no declara, y queda como advertencia para cuando se implemente el alta real de convenios.

ISP — Neutro. `IDescuento` ya era de un solo método antes de este trabajo; no hubo interfaz gorda que segregarse.

DIP — Tensionado pero compensado. Antes, `ServicioFacturacion.CalcularTotal(IFacturable, int)` dependía únicamente de una abstracción. La sobrecarga nueva, `CalcularTotal(IFacturable, int, Cliente)` (`ServicioFacturacion.cs:22-32`), agrega un parámetro de tipo **concreto**, `Cliente`, no `IDescuento` inyectado directamente. Es una tensión real: `ServicioFacturacion` ya no depende solo de abstracciones para esta sobrecarga.

Se compensa por dos razones. Primera, `Cliente` es una entidad de dominio, no un detalle técnico — la convención ya establecida en este sistema desde el Reto 1 es invertir dependencias hacia infraestructura (persistencia, autenticación, canal de notificación), no hacia entidades del propio dominio; `ServicioPuntos.AcumularPuntos(Cliente, int)` ya dependía de `Cliente` concreto desde el sistema original, sin que eso se haya señalado nunca como una violación. Segunda, la polimorfía real —qué algoritmo de descuento se ejecuta— sigue resuelta enteramente por `IDescuento`: `ServicioFacturacion` nunca pregunta qué tipo de descuento tiene el cliente, solo invoca la interfaz. El costo declarado es que este diseño ata la estrategia a la existencia de un `Cliente`; si en el futuro un descuento debiera aplicarse sin cliente identificado, esta sobrecarga no serviría y habría que agregar una tercera vía.

Vale la pena contrastar esto con P-02: ahí la estrategia se inyecta directo en el constructor del servicio (`ServicioPuntos(IREglaPuntos)`). Aquí se llega a la estrategia a través de un parámetro concreto (`cliente.Descuento`). La diferencia no es un error — responde a que la regla de puntos es una configuración del sistema (una promoción del día, igual para todos), mientras que el descuento es un dato de cada cliente (cada uno con su propio convenio) — pero es una asimetría real entre las dos aplicaciones de Strategy y conviene poder explicarla si preguntan por qué no se hizo igual en los dos casos.

Observer — P-08 (notificación)

SRP — Refuerza. Antes de este trabajo, `IServicioNotificacion` y `ServicioNotificacion` no tenían ningún consumidor: existían sin ejercer ninguna responsabilidad. `NotificacionArchivo` sí tiene una única razón de cambio —cómo se persiste una notificación— y es independiente de la razón de cambio del manejador de consola —cómo se colorea y formatea la salida—. Los dos observadores del mismo evento no se conocen entre sí ni comparten código.

OCP — Refuerza. El canal de archivo se agregó como una segunda suscripción a `EventoMovimiento.MovimientoRegistrado`, sin tocar la suscripción de consola que ya existía ni ninguna clase del dominio. Verificado en ejecución: la transcripción completa de una venta, comparada byte a byte contra el binario del sistema sin ninguno de los cuatro patrones, es **idéntica**; y `notificaciones.log` recibe el mensaje correctamente en cada venta.

LSP — Neutro. El evento no conoce el tipo `IServicioNotificacion`; cada suscriptor es una lambda independiente. No hay una jerarquía de tipos sustituyéndose aquí, así que no hay contrato de sustitución que evaluar.

ISP — Neutro. `IServicioNotificacion.EnviaNotificacion(string)` ya era de un solo método antes de que tuviera consumidores. No se segregó nada; se le dio uso real a algo que ya estaba bien segregado.

DIP — Refuerza. `Program.cs` declara `servicioNotificacion` como `IServicioNotificacion`, no como `NotificacionArchivo`; el canal concreto se decide en una sola línea del composition root. Cambiar de archivo a otro canal —o volver a `ServicioNotificacion`, que sigue existiendo como implementación alternativa sin cablear— no toca `ServicioMovimiento`, `EventoMovimiento` ni ninguna otra clase.

Verificación contra los errores típicos del enunciado

Error típico	¿Aplica a algún patrón adoptado?
Una fábrica que crece con un condicional por cada tipo nuevo	Sí — a Simple Factory en P-01. El switch de RepositorioProductos.cs:58-79 es exactamente este error. Se declara sin esconderlo, con su compensación de alcance en la fila de OCP: el eje de extensión de negocio (categorías nuevas de SC-1) no pasa por este switch, así que se acepta el costo residual de que una tercera <i>presentación</i> de medicamento sí lo reabrirla.
Una fachada que va absorbiendo lógica de negocio	No aplica — no se adoptó Facade en ninguno de los cuatro patrones.
Un punto de acceso global de instancia única (Singleton)	No aplica — no se adoptó Singleton.
Un método plantilla con pasos que alguna subclase no puede cumplir	No aplica — Template Method fue evaluado y descartado en la Actividad 2 (P-02).
Una envoltura que cambia el contrato de lo que envuelve	No aplica — Decorator fue evaluado y descartado dos veces (P-01 y P-02), precisamente para evitar este riesgo.

De los cinco errores típicos, uno sí aplica —a Simple Factory, el único de los cuatro patrones que no es de comportamiento resuelto por sustitución— y queda declarado y compensado, no oculto. Los otros cuatro no aplican a ninguno de los cuatro patrones adoptados.

Resumen de la matriz

De veinte celdas evaluadas (4 patrones × 5 principios): **8 Refuerza, 9 Neutro, 2 Tensionado pero compensado, 1 Roto.**

Dos tensiones reales declaradas: el OCP de Simple Factory en P-01 y el DIP de Strategy en P-03, ambas con su compensación y su costo explícitos. Una celda en Roto: el DIP de Simple Factory en P-01, declarado sin una compensación real que lo sostenga, con el arreglo de una línea que lo resolvería si el equipo decide aplicarlo.